

Pierflow Insurance Distribution — End-to-End Test Guide

Chapters 1–4 · Producer-side, Consumer-side, Money-side, Operations Version 1.0 · 2026-06-08

How to use this guide

This guide tests every meaningful path through the Pierflow Insurance Distribution platform — from registering an HMO through to an enrolled member, a paid claim, and a balanced ledger. It is designed for three audiences.

If you are a non-technical operator

- Skip **Part 1 (Setup)** unless you are also the person starting the local server.
- Read **Part 2** (UI walkthrough). Work the staff portal and the impersonated partner portal step by step.
- Use **Part 5** (Failure scenarios) when something does not behave as the guide says it should.
- Use **Part 6** (Sign-off checklist) when you are done.

If you are a technical QA

- Read **Part 1** if you are setting up the environment.
- Run **Part 3** (API walkthrough). It uses Postman + curl and mirrors the UI walkthrough at the protocol level.
- Use **Part 4** (Cross-checks) to verify ledger consistency + webhook delivery.
- Use **Part 5** to confirm error paths.

If you are the founder doing a self-test

- Read **Part 0.3** (the system map) first to confirm the mental model.
 - Use **Part 6** (Sign-off checklist) as your driving document; it cross-references every section.
 - Skim **Part 5** to make sure you remember each magic test string.
-

Part 0 — Orientation

0.1 What this platform does

Pierflow Insurance Distribution lets fintech apps embed health insurance from multiple HMOs through a single API. Three groups of people interact with it:

- **HMOs** (Reliance, Avon, Hygeia, etc.) supply plans, prices, and member onboarding.

- **EMR vendors** push the HMO catalogue into Pierflow and run the connector to the HMO.
- **Fintechs** browse the catalogue, quote plans for users, enroll members, debit wallets, and surface claims.

Pierflow sits in the middle: it stores the catalogue, computes personalised quotes, runs the enrollment lifecycle, instructs settlements, and keeps an independent ledger of every naira.

0.2 The four chapters

Chapter	Purpose	Key tables	Key API
1 — Producer side	HMO catalogue, AI-assisted normalisation, contracts, settlement preferences	HmoProvider , HmoPlan , HmoContract , ConnectorMapping	POST /v1/hmo-providers/:slug/plans
2 — Consumer side	Plan browsing, personalised quotes with frozen 24h pricing	HmoQuoteRequest , HmoQuote	GET /v1/plans , POST /v1/quotes
3 — Enrollment	Identity verification, payment orchestration, lifecycle webhooks	HmoEnrollment , IdentityVerification	POST /v1/enrollments
4 — Money side	Internal ledger, daily reconciliation, claims tracking, provider network	LedgerEntry , LedgerDiscrepancy , HmoClaim , HmoNetworkProvider	POST /v1/claims , GET /v1/providers

0.3 The end-to-end flow

The test we are about to run walks the full producer → consumer → money chain:

1. **Producer setup** (Chapter 1): Pierflow staff registers an HMO. The EMR vendor pushes the catalogue. Staff captures a Reliance-style contract.
2. **Consumer browse + quote** (Chapter 2): A fintech (impersonated by Pierflow staff) calls the catalogue API and submits a user profile. Pierflow returns five ranked, priced quotes with rationale.
3. **Enrollment** (Chapter 3): The fintech enrolls one user against the top quote. Pierflow verifies identity, freezes the splits snapshot, and waits for payment.
4. **Payment + settlement** (Chapter 3 + 4): The fintech confirms it debited the user's wallet and reports per-party credits inline. Pierflow advances the enrollment to ACTIVE, writes both INSTRUCTED and EXECUTED ledger entries, fires webhooks.
5. **Claim + tracking** (Chapter 4): The member visits a hospital. The fintech files a claim. Pierflow forwards to the HMO connector (stub), polls every 4h until terminal, fires webhooks on each transition.
6. **Reconciliation** (Chapter 4): Staff opens `/portal/reconciliation` to verify the ledger has zero discrepancies for the test enrollment.

0.4 Terms used throughout

- **kobo** — Nigerian minor unit. 1 naira = 100 kobo. All money in the API is in kobo. ₦8,500 = 850,000 kobo.
 - **wholesale** — the price the HMO charges Pierflow for a plan. Frozen at quote time.
 - **markup** — what the platform layer adds on top of wholesale. Frozen at quote time.
 - **member_pays** — wholesale + markup. What the user's wallet is debited.
 - **splits_snapshot** — the per-party breakdown of who receives what slice of member_pays. Frozen at quote time and used as the settlement instruction at enrollment time.
 - **INSTRUCTED entry** — a row in the ledger saying "Pierflow told the fintech to credit this much to this party."
 - **EXECUTED entry** — a row in the ledger saying "the fintech reported it actually credited this much to this party."
-

Part 1 — Setup (one-time)

This is for whoever is running the local server. Skip if someone else has set this up for you and you only need a browser.

1.1 Prerequisites

- **Node.js 24 or later**. Confirm with `node --version`.
- **npm** (bundled with Node).
- **A running Pierflow Postgres** (Supabase). The connection string in `.env.local` must work.
- **Environment variables** including `ANTHROPIC_API_KEY` (for AI-assisted plan normalisation, only needed if you test Chapter 1's mapping wizard), and the standard Pierflow secrets.

1.2 Start the server

From the project root:

```
npm run dev
```

Expected output:

```
▲ Next.js 14.2.35
- Local:   http://localhost:3000
- Environments: .env.local

✓ Starting...
✓ Ready in 3s
```

If you see `Failed to compile`, do not proceed. Capture the error and report it.

1.3 Sign in as Pierflow staff

1. Open `http://localhost:3000/portal`.
2. You will be redirected to the Clerk sign-in page.
3. Sign in with your usual Pierflow Platform email. The address must be in the `ADMIN_EMAILS` env variable for the staff portal to grant access.
4. You should land at the staff dashboard with the left navigation showing: Dashboard, Capture, Review queue, Patients, Partners, Customer orgs, HMO providers, Reconciliation, Settings.

If the left nav says only Records-related items, your account is not promoted to staff. Verify `ADMIN_EMAILS` includes the address you signed in with.

1.4 Install + open Postman

If you intend to run the API walkthrough (Part 3), install [Postman](#) and import the collection:

1. Open Postman.
2. **Import** → **File** → select
`c:\Users\oogunfuye\Documents\Pierflow\public\api\pierflow.postman_collection.json`.
3. Open the **Variables** tab of the imported collection.
4. Set `base_url` to `http://localhost:3000`.
5. Set `api_key` to a value we will mint in Part 2.

You can return to this once you have minted a key.

1.5 Connect a webhook receiver (optional but recommended)

To observe webhook deliveries:

1. Sign up for a free [webhook.site](#) account.
2. Copy the unique URL it gives you (looks like `https://webhook.site/12345abc-...`).
3. Keep that tab open. We will register this URL as a partner webhook endpoint in Part 2.5.

Part 2 — Non-technical UI walkthrough

This part runs the full Chapter 1 → Chapter 4 flow through the staff portal and the impersonated partner portal. No terminals, no code, no Postman. About 45 minutes end to end if everything works.

Section 2.1 — Register an HMO

You are Pierflow staff. You are onboarding "Reliance HMO" so its plans can be distributed.

1. From the staff dashboard, click **HMO providers** in the left nav.
2. You should see the HMO providers list page. It may be empty if you have not registered any before.
3. Click the **+ New HMO provider** button at the top right.
4. Fill the form:

- **Slug:** reliance-hmo
- **Display name:** Reliance HMO
- **Registration number** (optional): NAICOM-2026-001
- **Contact email** (optional): ops@reliancehmo.example
- **Default settlement mode:** choose **In fintech account** (the cleaner option for our test).

5. Click **Save**.

Expected: You land on `/portal/hmo-providers/reliance-hmo`. The header shows "Reliance HMO" with a **Pending** chip. The right sidebar shows the slug, contact, settlement preference. The middle of the page is empty under "Plans" and "Contracts" — we have not added either yet.

Flag this if: You see an error, the slug field complains about duplicates, or the redirect lands somewhere unexpected.

Section 2.2 — Save a plan mapping (AI-assisted normalisation)

We will use the mapping wizard to teach Pierflow how to translate Reliance's native plan format into Pierflow's Universal Plan Schema. The output of this section is a saved + activated **mapping**, not a plan. Plans get inserted later (Section 2.4 or 3.2) using this mapping.

1. Still on `/portal/hmo-providers/reliance-hmo`, click **Plan mapping** in the action row.
2. You land on `/portal/hmo-providers/reliance-hmo/mapping`. There are three steps shown at the top: **Provide sample, Review proposal, Activate**.
3. In the "Paste a single native plan" text area, paste the following synthetic Reliance plan:

```
{
  "plan_id": "REL-SILVER-IND",
  "plan_name": "Silver Plan",
  "scope": "individual",
  "billing": "monthly",
  "individual_monthly_naira": 8500,
  "age_bands": [
    { "min_age": 18, "max_age": 35, "monthly_naira": 8500 },
    { "min_age": 36, "max_age": 50, "monthly_naira": 11000 }
  ],
  "coverage": {
    "outpatient": { "covered": true, "limit_naira": 200000, "co_pay_percent": 0 },
    "inpatient": { "covered": true, "limit_naira": 1000000, "co_pay_percent": 10 },
    "maternity": { "covered": true, "limit_naira": 300000, "waiting_period_days": 270 },
    "dental": { "covered": false },
    "telemedicine": { "covered": true, "unlimited": true }
  },
  "exclusions": ["HIV/AIDS treatment", "Cosmetic surgery"],
  "waiting_periods": { "general_days": 30, "maternity_days": 270 }
}
```

4. Click **Propose mapping with AI**. This calls Anthropic Claude Haiku 4.5. Expect 3–8 seconds of latency. The button shows a spinner.

Expected: A proposal table appears. Each row maps a Universal Plan Schema field on the left (e.g. `external_id`) to a JSONPath on the right (`plan_id`). Each row has a **confidence** percentage. The summary at the top shows "Average confidence: 78%" or similar. Notes from the model explain assumptions (e.g. "naira values were converted to kobo by multiplying by 100").

Flag this if: The request fails (`ANTHROPIC_API_KEY` is missing), the confidence is below 60% (indicates a bad sample), or any row has empty rationale.

5. Click **Activate this mapping**.

Expected: The page shows the mapping in **Active** state. There is now a saved `ConnectorMapping` row that future native catalogue pushes will use to translate.

Section 2.3 — Author a contract

This is where you tell Pierflow what each party gets paid when a user buys this HMO's plans.

1. From `/portal/hmo-providers/reliance-hmo`, click **Contracts** in the action row.
2. You land on `/portal/hmo-providers/reliance-hmo/contracts`. Click **+ New contract**.
3. The contract wizard has 4 steps: **Mode**, **Parties**, **Terms**, **Review**.

Step 1 — Mode

You see three cards: "Gross share", "Wholesale + fixed markup", "Wholesale + per-party markup". Read each one-line description. Pick **Wholesale + fixed markup** for our test. Click **Continue**.

Step 2 — Parties

The wizard pre-fills three rows: Pierflow ₦200, EMR vendor ₦300, Fintech ₦1,000. Every row's "When" is **Both events** by default.

- Leave all three rows as-is.
- The bottom-right shows the sum of percentage shares. In `MARKUP_FIXED` mode this is hidden (correct).
- Click **Continue**.

Step 3 — Terms

- **Effective from:** today.
- **Markup amount:** ₦1,500 (this is the sum of the three flat lines).
- **Enrollment fee:** ₦500 (optional one-time fee for our test).
- **Enrollment fee goes to:** a new dropdown appears because we set enrollment fee > 0 in a markup mode. Pick **FINTECH** (the fintech keeps the signup fee — common arrangement).
- **Remainder bearer:** leave as **FINTECH**.
- Click **Continue**.

Step 4 — Review

A preview shows what the user pays at various sample wholesale amounts. With wholesale = ₦8,500 and markup = ₦1,500, you should see:

Line	Amount
Wholesale (to HMO)	₦8,500
+ Markup	₦1,500
= Member pays	₦10,000
Pierflow	₦200
EMR_VENDOR	₦300
FINTECH	₦1,000 (remainder)

A separate enrollment-fee preview shows the full ₦500 going to FINTECH.

- Click **Activate this contract**.

Expected: You return to `/portal/hmo-providers/reliance-hmo/contracts` and see a contract row in **Active** state.

Flag this if: The math does not match the table above, the preview is missing the wholesale + markup header in markup modes, or any row shows "capped (raw ...)" annotations on this clean example.

Section 2.4 — Activate the HMO and publish a sample plan

Back on `/portal/hmo-providers/reliance-hmo`, the Pending chip should still be visible.

1. Click **Activate** in the top-right. The chip changes to **Active**.
2. The Plans section shows an empty state with a **Publish a sample plan** button. Click it.
3. Within ~2 seconds, the button reads "Sample plan created" and the page refreshes. The Plans section now lists **Silver Plan (sample)** with a wholesale of ₦8,500 and status ACTIVE.

This button uses the mapping you activated in Section 2.2 to translate a synthetic Reliance plan from native format into Pierflow's Universal Plan Schema and inserts it. Re-running the button updates the same plan in place; it's idempotent and safe to click multiple times.

Expected: The Plans count in the right sidebar increments to 1. Clicking the **Silver Plan (sample)** row opens the plan detail page at `/portal/hmo-providers/reliance-hmo/plans/{planId}`. That page renders the full Universal Plan Schema produced by the AI normalisation: pricing (₦8,500 individual + 4 age bands), the coverage table (outpatient ₦200K, inpatient ₦1M w/ 10% co-pay, maternity ₦300K w/ 270d wait, dental not covered, optical ₦30K, emergency ₦500K, telemedicine unlimited), waiting periods (general 30d, maternity 270d, pre-existing 365d), and three exclusions.

Use this page to **verify the AI translation is accurate** — every value should reflect the wizard sample you pasted in Section 2.2. The sidebar shows freshness timestamps and a recent activity log (a single "partial update" entry from the publish click).

Flag this if: The button shows "Couldn't publish — usually means there's no active mapping yet" (go back to Section 2.2 and confirm the mapping is ACTIVE, not DRAFT); the plan publishes but a coverage row shows the wrong limit (the mapping translated incorrectly — Reset the mapping and re-propose); or wholesale is not ₦8,500.

Section 2.5 — Register a fintech partner

We will simulate a fintech signing up via the public flow.

1. Open a new private/incognito browser tab.
2. Navigate to `http://localhost:3000/get-started`.
3. **Step 1 — Who are you?** Pick **Fintech, bank, or super-app**.
4. **Step 2 — What are you doing?**
 - Primary use case: **Embed HMO health insurance in our app**.
 - Expected volume: **1,000 – 10,000 records / month**.
 - Timeline: **Integrating in 1–3 months**.
5. **Step 3 — Your details.**
 - Full name: any name.
 - Email: use a real Gmail address you can read.
 - Company: **Test Fintech**.
6. Click **Submit application**.

Expected: A confirmation screen. Within 1 minute, an email arrives from Pierflow with an invitation link.
Do not click the invitation yet.

Flag this if: The email never arrives. Check `ADMIN_EMAILS` and Gmail credentials in `.env.local`.

Now switch back to your staff browser tab.

7. In the staff portal, click **Partners** in the left nav.
8. The new "Test Fintech" row appears in **Pending sandbox** state with an audience chip.
9. Click into the row. You land on `/portal/partners/{id}`.
10. The right sidebar shows: type **FINTECH**, consumes **INSURANCE**, no API keys yet.
11. Click **Approve sandbox access** at the top right.
12. The Partner now shows **Sandbox** state.

Section 2.6 — Impersonate the fintech

This is the feature that lets one person test both producer and consumer sides without juggling accounts.

1. From the partner detail page, click **View as Test Fintech** at the top.
2. The page reloads. A **persistent amber banner** appears at the top: *"Viewing portal as Test Fintech. Every action you take is logged as staff_impersonating ({your email}). [End session]"*
3. The left navigation has switched to the **partner** layout: Overview, Organizations, API keys, Webhooks, Usage, Docs.

You are now acting as the fintech for testing purposes. Every action you take is recorded as `staff_impersonating` in the audit.

Section 2.7 — Mint an API key (as the fintech)

1. Click **API keys** in the left nav.
2. Click + **Generate new key**.
3. Pick a label: `Test key`.
4. Click **Generate**.

Expected: A modal shows the full key once: `pf_test_sk_...`. **Copy it now.** Save it somewhere — you cannot view it again. Set this aside for Part 3 (Postman). The key automatically carries `insurance:read` and `insurance:write` scopes because the partner consumes the INSURANCE product.

Flag this if: The scope chips below the key do not show `insurance:read` and `insurance:write`.

Section 2.8 — Register a webhook endpoint (as the fintech)

If you set up `webhook.site` in Part 1.5:

1. Click **Webhooks** in the left nav.
2. Click + **Register endpoint**.
3. URL: paste your `webhook.site` URL.
4. Events: check all `hmo_enrollment.*` and `hmo_claim.*` events.
5. Click **Register**.

The endpoint appears in the list with a generated HMAC secret. Keep `webhook.site` open in another tab so you can watch deliveries arrive.

Section 2.9 — End impersonation

Click **End session** on the amber banner. You return to `/portal/partners` as staff. The `ImpersonationSession` audit row is now closed (you can verify it later with a direct database query if needed).

Section 2.10 — Verify the impersonation audit trail

The audit trail confirms every staff action during impersonation is attributable.

1. Open a DB query tool (Supabase Studio, TablePlus, etc.) against your Pierflow database.
2. Run: `SELECT id, "staffEmail", "partnerName", "startedAt", "endedAt" FROM impersonation_sessions ORDER BY "startedAt" DESC LIMIT 5;`
3. You should see one row whose `endedAt` is approximately now, with your staff email and `partnerName` = `Test Fintech`.

Flag this if: `endedAt` is null (session never closed), or `staffEmail` does not match yours.

Note on Pierflow's SQL conventions: tables are `snake_case` (e.g. `impersonation_sessions`) but columns are `camelCase` (e.g. `staffEmail`). Postgres requires double-quotes around mixed-case

identifiers, so all column references in this guide use `"camelCase"`. Without the quotes, Postgres lower-cases the identifier and the query fails with `column "staffemail" does not exist`.

Part 3 — Technical API walkthrough

This part runs the same end-to-end flow as Part 2 but through the API directly. It assumes you have completed sections 2.1, 2.3, 2.4, 2.5, and 2.7 to get a registered HMO, an active contract, and an API key. You will need the `pf_test_sk_...` API key from section 2.7.

Open Postman. The relevant folders for this section are **11 · Plans**, **12 · Quotes**, and **13 · Enrollments**. Folders 1–10 are the Records API surface and aren't exercised here. Run the three insurance folders top-to-bottom; the test scripts in each request auto-populate the variables the next folder needs (`plan_id` → `quote_id` → `enrollment_id`).

Section 3.1 — Set Postman variables

Open the collection's Variables tab. Confirm:

- `base_url` : `http://localhost:3000`
- `api_key` : the `pf_test_sk_...` from section 2.7
- `provider_slug` : `reliance-hmo`
- `fintech_user_ref` : pick anything memorable, e.g. `user_test_001`

The other variables (`plan_id`, `quote_id`, `enrollment_id`, etc.) will auto-populate as you run requests.

Section 3.2 — Push the catalogue

Important clarification: The mapping wizard from Section 2.2 only saves the *translation rule* — it does not insert any plans. To actually publish a Silver Plan, you need a separate step.

The real catalogue-push endpoint is `POST /v1/hmo-providers/:slug/plans`. It's not exposed in the Postman collection because partners need an explicit `PartnerOrganizationLink` to the HMO to call it, and the test fintech you minted in Section 2.7 is a *consumer* of plans, not a publisher. In production, this endpoint is called by an EMR vendor's connector.

For testing, use the "Publish a sample plan" button instead.

1. Navigate to **HMO providers** → **Reliance HMO** in the staff portal.
2. If the Plans section shows "No plans yet", click the **Publish a sample plan** button at the bottom of that panel.
3. If you've already published once and want to add more, click the **Publish a sample plan** chip in the top action row (next to "Contracts").

The button uses the active mapping (the one you activated in Section 2.2) to translate a synthetic Silver Plan from EMR-vendor native format into Pierflow's Universal Plan Schema, then inserts it. It exercises the exact same code path as a real EMR vendor's catalogue push.

Expected: The page refreshes within ~2 seconds and the Plans section shows the **Silver Plan (sample)** with a wholesale of ₦8,500 and status ACTIVE. The button text changes to "Sample plan created" or "Sample plan updated".

Flag this if: The button shows "Couldn't publish — usually means there's no active mapping yet" (run Section 2.2 again), or the plan appears but its status is not ACTIVE.

Once the Silver Plan is visible, the rest of Part 3 will work.

Section 3.3 — Browse the catalogue (read-side)

Folder **11 · Plans** → **List plans**. Click **Send**.

Expected (200): An `items` array containing at least one plan. The first item should be Silver Plan with:

- `id` (the same as `plan_id`)
- `hmo.slug` = `reliance-hmo`
- `pricing.individual_monthly` = `850000` (kobo)
- `is_stale` = `false`

Try the filter variants:

- `Plans` → `List plans (state filter)` — try `state=Lagos`.
- `Plans` → `List plans (budget filter)` — pass `max_monthly_premium_ngn=1000000`. The Silver Plan (₦8,500) should appear.
- Set `max_monthly_premium_ngn=500000` (₦5,000). The Silver Plan should be filtered out.

Flag this if: Filters do not narrow the result set, or `last_synced_at` is missing.

Section 3.4 — Submit a quote

Folder **12 · Quotes** → **Create quote**.

Body:

```
{
  "age_in_years": 28,
  "sex": "M",
  "dependents": 0,
  "monthly_budget_ngn": 1000000,
  "state": "Lagos",
  "lga": "Ikeja",
  "limit": 5,
  "fintech_ref": "session_test_001"
}
```

Click **Send**.

Expected (200):

```
{
  "request_id": "qreq...",
  "expires_at": "<24h from now ISO>",
  "quotes": [
    {
      "id": "quote...",
      "plan_id": "plan...",
      "rank": 1,
      "score": <number between 0.7 and 0.9>,
      "wholesale_ngn": "850000",
      "markup_ngn": "150000",
      "member_pays_ngn": "1000000",
      "rationale": {
        "reasons": ["Fits your ₦10,000 monthly budget", ...],
        "warnings": [],
        "signals": {...}
      },
      "contract_version": 1,
      "splits_snapshot": {...},
      "expires_at": "<same as top-level>"
    }
  ]
}
```

The test script auto-populates `quote_id` and `quote_request_id`. Inspect the **splits_snapshot** carefully:

```
{
  "mode": "MARKUP_FIXED",
  "wholesale_ngn": "850000",
  "markup_ngn": "150000",
  "member_pays_ngn": "1000000",
  "hmo_line": { "role": "HMO", "amount_ngn": "850000", "settlement_tag": null },
  "lines": [
    { "role": "PIERFLOW", "amount_ngn": "20000", "settlement_tag": "pierflow:platform_fee", "is_re"},
    { "role": "EMR_VENDOR", "amount_ngn": "30000", "settlement_tag": "emr_vendor:default", "is_re"},
    { "role": "FINTECH", "amount_ngn": "100000", "settlement_tag": "fintech:self", "is_remainder"}
  ]
}
```

Flag this if: `score` is below 0.5 (something is off in personalisation), `splits_snapshot.lines` is empty, or amounts in `lines` do not sum to `markup_ngn`.

Section 3.5 — Retrieve the quote

Folder **12 • Quotes** → **Retrieve quote (by quote id)**. Click **Send**.

Expected: The same payload as the single quote you just received. Money fields are strings ("850000") for BigInt safety.

Section 3.6 — Enroll a member (happy path)

Folder **13 • Enrollments** → **Create enrollment**.

Body (use a fresh `idempotency_key` to avoid replays):

```
{
  "plan_id": "{{plan_id}}",
  "quote_id": "{{quote_id}}",
  "fintech_user_ref": "{{fintech_user_ref}}",
  "idempotency_key": "enr_test_001",
  "identity": {
    "nin": "12345678901",
    "full_name": "Adaeze Margaret Nwosu",
    "date_of_birth": "1995-03-14",
    "sex": "F",
    "phone": "+2348012345678"
  }
}
```

Click **Send**.

Expected (202):

```
{
  "enrollment": {
    "id": "enr_...",
    "status": "PENDING_PAYMENT",
    "wholesale_ngn": "850000",
    "markup_ngn": "150000",
    "member_pays_ngn": "1000000",
    "splits_snapshot": {...},
    "hmo_policy_id": null
  },
  "identity": {
    "status": "AUTO_APPROVED",
    "confidence": 95,
    "provider": "STUB"
  },
  "idempotent_replay": false
}
```

The test script populates `enrollment_id` . Two webhooks should arrive at `webhook.site` within seconds:

- `hmo_enrollment.created`
- `hmo_enrollment.identity_verified` (with `identity_status: "AUTO_APPROVED"`)

Flag this if: Status is anything but `PENDING_PAYMENT` , identity confidence is below 90 for a clean name, or no webhooks arrive within 30 seconds.

Section 3.7 — Confirm payment received (with executed credits)

Folder **13 · Enrollments** → **Confirm payment received**.

This step does double duty: it advances the enrollment to *ACTIVE* *and* reports the executed credits so the ledger has both sides for reconciliation.

Body:

```
{
  "amount_ngn": "1000000",
  "payment_ref": "wallet_txn_test_001",
  "executed_credits": [
    { "role": "HMO", "settlement_tag": null, "amount_ngn": "850000" },
    { "role": "PIERFLOW", "settlement_tag": "pierflow:platform_fee", "amount_ngn": "20000" },
    { "role": "EMR_VENDOR", "settlement_tag": "emr_vendor:default", "amount_ngn": "30000" },
    { "role": "FINTECH", "settlement_tag": "fintech:self", "amount_ngn": "100000" }
  ]
}
```

Click **Send**.

Expected (200): The enrollment is now `ACTIVE` , with `hmo_policy_id` populated by the stub connector (looks like `STUB-REL-SILVER-IND-{4 hex chars}`).

Webhook deliveries at `webhook.site` within 10 seconds:

- `hmo_enrollment.payment_received`
- `hmo_enrollment.submitted_to_hmo`
- `hmo_enrollment.activated`

Flag this if: Status is anything but `ACTIVE` , `hmo_policy_id` is null, or any of the three webhooks fail to arrive.

Section 3.8 — Submit a claim

Folder **13 • Enrollments** → (no direct option) → switch to **POST /v1/claims** in the request builder.

Body:

```
{
  "enrollment_id": "{{enrollment_id}}",
  "fintech_user_ref": "{{fintech_user_ref}}",
  "service_date": "2026-06-08",
  "service_type": "outpatient_visit",
  "facility_name": "Reddington Hospital – Ikeja",
  "amount_ngn": "2500000",
  "diagnosis_codes": ["J45.9"],
  "procedure_codes": [],
  "notes": "Routine asthma follow-up",
  "idempotency_key": "clm_test_001"
}
```

Click **Send**.

Expected (202):

```
{
  "claim": {
    "id": "clm...",
    "status": "PENDING_HMO",
    "hmo_claim_id": "STUB-CLM-{4 hex chars}",
    "amount_ngn": "2500000"
  },
  "idempotent_replay": false
}
```

Webhooks arrive: `hmo_claim.submitted` .

Section 3.9 — Watch the claim progress

The poll-claims cron normally runs every 4 hours. To exercise it now, hit the cron endpoint manually:

```
GET /v1/cron/poll-claims
Authorization: Bearer <your CRON_SECRET>
```

Run that request once. Wait 15 seconds. Run it again. Wait 15 seconds. Run it a third time.

Expected progression:

- After 1st run: `hmo_claim.under_review` webhook fires; claim status → `UNDER_REVIEW` .
- After 2nd run: `hmo_claim.approved` webhook fires; claim status → `APPROVED` .
- After 3rd run: `hmo_claim.paid` webhook fires; claim status → `PAID` .

Retrieve the claim to confirm: `GET /v1/claims/{{claim_id}}` . Expected `status: "PAID"` .

Flag this if: The status does not advance, or a webhook fires twice for the same transition.

Section 3.10 — Browse the provider network

Folder is yet to be added to Postman in this version; use curl or build the request manually.

```
GET /v1/providers?state=Lagos&type=HOSPITAL
Authorization: Bearer <your api_key>
```

Expected (200): An `items` array containing at least Reddington (Ikeja) and Lagoon (Victoria Island).

```
GET /v1/providers?specialty=cardiology
```

Should return Reddington, Lagoon, and Abuja National (the three with `cardiology` in their specialties).

Part 4 — Cross-checks

This part verifies the money side is consistent and the audit trail is honest.

Section 4.1 — Ledger consistency

After Section 3.7 (enrollment activated with executed credits), the ledger should have a balanced set for our test enrollment.

Open your DB query tool and run:

```
SELECT
  "accountId",
  kind,
  "amountNgn",
  source
FROM ledger_entries
WHERE "correlationId" = '<your enrollment_id from 3.6>'
ORDER BY kind, "occurredAt";
```

Expected:

- 4 rows with `kind = 'INSTRUCTED'` (one per party + a negative user row).
- 4 rows with `kind = 'EXECUTED'` (matching).
- `SUM("amountNgn") WHERE kind = 'INSTRUCTED'` must be exactly 0.
- `SUM("amountNgn") WHERE kind = 'EXECUTED'` must be exactly 0.

If either kind does not sum to 0, the ledger is mis-balanced. **Flag this immediately**; this is the single most important invariant in the platform.

Section 4.2 — Reconciliation

Run the reconciliation cron manually:

```
GET /v1/cron/reconcile-settlements
Authorization: Bearer <your CRON_SECRET>
```

Expected (200):

```
{
  "ok": true,
  "elapsed_ms": <number>,
  "scanned": >= 1,
  "balanced": >= 1,
  "discrepancies": 0,
  "pending": 0,
  "errors": 0
}
```

If `discrepancies > 0`, open `/portal/reconciliation` in the staff portal to investigate. The Open tab will show the affected enrollment.

Section 4.3 — Reconciliation UI

In the staff portal, navigate to **Reconciliation** in the left nav.

Expected: If everything is healthy, all tabs are empty except possibly the **Resolved** tab (showing your auto-resolved enrollment).

If a row appears in **Open**, click into it:

- The per-account breakdown table shows Instructed, Executed, Delta for each ledger account.
- A non-zero Delta column highlights the discrepant account in red.
- Action buttons let you Acknowledge, Mark resolved, or Write off.
- A re-reconcile button retries the comparison after you've manually intervened.

To simulate a discrepancy:

1. Manually update one ledger entry's amount in the DB.
2. Re-run the cron.
3. The enrollment now appears in **Open** with the right delta.
4. Reverse the change and re-reconcile; the row should move to **Resolved**.

Section 4.4 — Webhook deliveries audit

If you set up `webhook.site`:

- Open the `webhook.site` tab.
- You should see, in order:
 1. `hmo_enrollment.created`
 2. `hmo_enrollment.identity_verified`
 3. `hmo_enrollment.payment_received`
 4. `hmo_enrollment.submitted_to_hmo`
 5. `hmo_enrollment.activated`
 6. `hmo_claim.submitted`
 7. `hmo_claim.under_review`
 8. `hmo_claim.approved`
 9. `hmo_claim.paid`

Each delivery carries an `X-Pierflow-Signature` header. The payload is JSON with `event`, `emitted_at`, `partner_id`, and `data`.

Flag this if: Any event is missing, duplicated, or fires in the wrong order.

Section 4.5 — Audit trail review

Pierflow stores append-only event logs for every state machine. Verify them:

```
SELECT kind, "fromStatus", "toStatus", "occurredAt"
FROM hmo_enrollment_events
WHERE "enrollmentId" = '<your enrollment_id>'
ORDER BY "occurredAt";
```

Expected events: `CREATED` → `IDENTITY_VERIFIED` → `PAYMENT_RECEIVED` → `SUBMITTED_TO_HMO` → `HMO_APPROVED` → `ACTIVATED` .

```
SELECT kind, "fromStatus", "toStatus", "occurredAt"
FROM hmo_claim_events
WHERE "claimId" = '<your claim_id>'
ORDER BY "occurredAt";
```

Expected events: `SUBMITTED` → `SENT_TO_HMO` → `STATUS_POLLED` (possibly multiple) → `UNDER_REVIEW` → `APPROVED` → `PAID` .

Part 5 — Failure scenarios

The platform must handle bad input cleanly. Run these to confirm.

Section 5.1 — Identity verification rejection

Repeat Section 3.6 but use a magic name to force a rejection:

```
{
  "plan_id": "{{plan_id}}",
  "fintech_user_ref": "user_test_002",
  "idempotency_key": "enr_test_002",
  "identity": {
    "nin": "12345678901",
    "full_name": "TEST_FAIL synthetic user",
    "date_of_birth": "1995-03-14"
  }
}
```

Expected (422):

```
{
  "error": "IDENTITY_REJECTED",
  "detail": "Identity check returned 30% confidence (below 60 threshold)."
}
```

Webhook arrives: `hmo_enrollment.identity_rejected` with `enrollment_id: null` . No `HmoEnrollment` row was created (verify with `SELECT count(*) FROM hmo_enrollments WHERE "fintechUserRef" = 'user_test_002';` → 0).

Section 5.2 — Identity soft review

Use the other magic name:

```
{
  "plan_id": "{{plan_id}}",
  "fintech_user_ref": "user_test_003",
  "idempotency_key": "enr_test_003",
  "identity": {
    "nin": "12345678901",
    "full_name": "TEST_SOFT synthetic user",
    "date_of_birth": "1995-03-14"
  }
}
```

Expected (202): Enrollment is created (status = `PENDING_PAYMENT`) with `identity.status = "SOFT_REVIEW"` and `identity.confidence = 70`.

This path means "policy issued but flagged for manual review by Pierflow ops."

Section 5.3 — Quote expired

Quotes expire after 24 hours. To simulate:

1. Submit a quote (Section 3.4).
2. Manually update its `expiresAt` to a past timestamp:

```
UPDATE hmo_quotes SET "expiresAt" = NOW() - INTERVAL '1 hour' WHERE id = '<your quote_id>';
```

3. Try to enroll using that expired `quote_id`.

Expected (404):

```
{
  "error": "QUOTE_NOT_FOUND_OR_EXPIRED",
  "detail": "Quote expired at <ISO timestamp>"
}
```

Section 5.4 — Idempotent enrollment replay

Run Section 3.6 again with the **same** `idempotency_key`. **Do not change anything else.**

Expected (202): The same enrollment object is returned. The new field `idempotent_replay: true` indicates the response is a replay. Critically: no new ledger entries, no new webhooks fire.

Section 5.5 — Amount mismatch on payment-received

Repeat Section 3.7 but send `amount_ngn: "999999"` (1 kobo less than `member_pays_ngn`).

Expected (422):

```
{
  "error": "AMOUNT_MISMATCH",
  "detail": "Expected 1000000 kobo, received 999999 kobo."
}
```

The enrollment remains in `PENDING_PAYMENT`. No state transition.

Section 5.6 — Stale data hint

The Postman test for **List plans** verifies the `is_stale` field on each item.

The synthetic Silver Plan's `staleAfter` is 26 hours after the push (we set it on the catalogue ingest endpoint). To simulate stale data:

```
UPDATE hmo_plans SET "staleAfter" = NOW() - INTERVAL '1 hour' WHERE id = '<your plan_id>';
```

Re-run **GET /v1/plans**. The plan should now have `is_stale: true`. The plan still appears in the list — but the fintech client is expected to surface a "data may be stale" hint.

Reset:

```
UPDATE hmo_plans SET "staleAfter" = NOW() + INTERVAL '24 hours' WHERE id = '<your plan_id>';
```

Section 5.7 — Wrong scope

Mint a second API key, but for the EMR vendor partner (not the fintech). EMR vendor keys carry only `records:read`. Try to call `GET /v1/plans` with it.

Expected (403):

```
{
  "error": "INSUFFICIENT_SCOPE",
  "detail": "This API key does not have the 'insurance:read' scope."
}
```

This confirms the new scope guard works correctly.

Part 6 — Sign-off checklist

Run this at the end. Tick each box. If any item fails, do not sign off — return to the relevant section above.

Chapter 1 — Producer side

- ☐ HMO can be registered in the staff portal
- ☐ HMO status transitions: Pending → Active
- ☐ AI mapping wizard returns a proposal with confidence $\geq 70\%$
- ☐ Saved mapping can be activated
- ☐ Native catalogue push translates through the mapping
- ☐ Contract wizard supports all three modes (Gross share, Markup fixed, Markup from shares)
- ☐ Markup-mode contract requires enrollment beneficiary when fee > 0
- ☐ Splits preview is mathematically consistent (parts sum to total)

Chapter 2 — Consumer side

- ☐ `GET /v1/plans` returns active plans
- ☐ Filters narrow correctly (state, max budget, age, scope)
- ☐ `POST /v1/quotes` returns top-N ranked quotes
- ☐ Quote rationale contains both `reasons` and `signals`
- ☐ `splits_snapshot.lines` sums equal `markup_ngn`
- ☐ Quotes expire after 24h (Section 5.3)
- ☐ Wrong scope rejected (Section 5.7)

Chapter 3 — Enrollment

- ☐ Happy path: PENDING_PAYMENT → PENDING_HMO → ACTIVE
- ☐ Three webhooks fire on the happy path
- ☐ Identity rejection: no enrollment row, single webhook (Section 5.1)
- ☐ Identity soft review: enrollment created, identity flagged (Section 5.2)
- ☐ Idempotency replay returns existing enrollment unchanged (Section 5.4)
- ☐ Amount mismatch rejected (Section 5.5)
- ☐ Enrollment cancellation works end to end

Chapter 4 — Money side

- ☐ Ledger has 8 entries for happy-path enrollment (4 instructed + 4 executed)
- ☐ Both sums equal 0 kobo
- ☐ Reconciliation cron returns 0 discrepancies for a balanced enrollment
- ☐ Manually injected discrepancy surfaces in `/portal/reconciliation`
- ☐ Claim submission returns PENDING_HMO with stub claim id
- ☐ Polling cron advances claim through UNDER_REVIEW → APPROVED → PAID

- ☐ All four claim webhook events fire
- ☐ GET /v1/providers returns synthetic Lagos / Abuja hospitals
- ☐ Provider filter by state, type, specialty narrows correctly

Cross-cutting

- ☐ Impersonation banner is sticky on every page during the session
- ☐ Impersonation session shows up in impersonation_sessions with non-null ended_at
- ☐ Webhook signatures (X-Pierflow-Signature) are present on every delivery
- ☐ No console.error lines appear in the server logs during the happy path
- ☐ Total time for the happy path (Sections 3.2 → 3.7) is under 90 seconds end to end

Sign-off

- Tester name: _____
 - Date: _____
 - Network used (local / staging / production): _____
 - Notes:
-

Appendix A — Sandbox magic strings

Where	String	Effect
identity.full_name on enrollment	contains TEST_FAIL	Identity rejected (30% confidence)
identity.full_name on enrollment	contains TEST_SOFT	Identity soft review (70% confidence)
Any other name	(default)	Identity auto-approved (95% confidence)
notes on claim	contains test_reject	Claim rejected on next poll
notes on claim	contains test_fast_approve	Claim approved on next poll
notes on claim	(default)	Claim walks SUBMITTED → UNDER_REVIEW → APPROVED → PAID across three polls

Appendix B — Common errors and remedies

Error	Likely cause	Remedy
Failed to compile on <code>npm run dev</code>	Schema drift or stale Prisma client	Re-run <code>npx prisma generate && npm run dev</code>
P1001 on Prisma commands	Network / Supabase pooler unreachable	Switch network or check Supabase dashboard
INSUFFICIENT_SCOPE 403	API key was issued before scopes were defaulted	Mint a new key from <code>/portal/keys</code>
Webhook never arrives	The webhook endpoint is unreachable or your <code>webhook.site</code> URL expired	Re-register with a fresh URL
Identity check returned 30% and you did not use magic name	Stub provider is throwing — check <code>IDENTITY_HASH_SECRET</code> is set	Set the env var and restart the server
Mapping wizard returns "No proposal"	<code>ANTHROPIC_API_KEY</code> is missing	Add to <code>.env.local</code> and restart
Reconciliation cron 401	<code>CRON_SECRET</code> env var not set	Set it in <code>.env.local</code> ; pass as Bearer token

Appendix C — Reference URLs

Surface	URL
Staff dashboard	http://localhost:3000/portal
HMO providers list	http://localhost:3000/portal/hmo-providers
Partners inbox	http://localhost:3000/portal/partners
Reconciliation queue	http://localhost:3000/portal/reconciliation
Public onboarding	http://localhost:3000/get-started
Partner portal (post-login)	http://localhost:3000/portal/overview
API docs	http://localhost:3000/docs/quickstart/introduction
Postman collection	http://localhost:3000/docs/api/postman
Reconcile cron	http://localhost:3000/v1/cron/reconcile-settlements
Poll-claims cron	http://localhost:3000/v1/cron/poll-claims

Appendix D — Test data summary

For a single happy-path run, the following data is created:

- 1 `HmoProvider` (Reliance HMO)
- 1 `ConnectorMapping` (Active)
- 1 `HmoPlan` (Silver Plan)
- 1 `HmoContract` (Active, MARKUP_FIXED)
- 1 `Partner` (Test Fintech, Sandbox)
- 1 `PartnerApiKey`
- 1 `WebhookEndpoint`
- 1 `HmoQuoteRequest`
- 5 `HmoQuote` rows (top-N from the request)
- 1 `HmoEnrollment` (ACTIVE)
- 1 `IdentityVerification`
- 6 `HmoEnrollmentEvent` rows
- 8 `LedgerEntry` rows (4 instructed + 4 executed)
- 0 `LedgerDiscrepancy` rows (balanced)
- 1 `HmoClaim` (PAID)
- 4 `HmoClaimEvent` rows
- ≥ 6 `HmoNetworkProvider` rows (seeded synthetic)
- 1 `ImpersonationSession` (closed)
- 9 webhook deliveries

To clean up for a re-test, use the cleanup script:

```
node --env-file=.env.local scripts/cleanup-demo-state.mjs --drop-all-partners --keep-orgs=pierflow
```

This drops every Partner and resets transient data. The HMO provider, plan, contract, and mapping can be re-used; just register a fresh fintech.

End of guide. Total length: roughly 35 printed pages.